

A HANDS-ON TUTORIAL · SPRING AI 2.0

Building Agents with Spring AI

From your first `ChatClient` call to a hierarchy of specialised subagents — seven recipes, in order of difficulty, each one runnable on its own.

THE SEVEN RECIPES

- 1 The conversational agent — memory and instructions
- 2 The tool-using agent — reasoning and acting
- 3 The grounded agent — answering from your own data
- 4 The routing agent — picking the right specialist
- 5 The orchestrator — decomposing work across parallel workers
- 6 The self-correcting agent — generate, critique, refine
- 7 The delegating agent — subagents with isolated context

Targets Spring AI 2.0.1 on Spring Boot 4.0/4.1, Java 17+

Audience Java developers new to building AI agents

Prerequisites Spring Boot basics and an API key for one model provider

Sources spring.io/projects/spring-ai, docs.spring.io/spring-ai, habuma.com/springairecipes

Contents

PART I — FOUNDATIONS

1 What an Agent Actually Is

- 1.1 Three things people call "an agent"
 - 1.2 The agent loop
 - 1.3 Workflow or agent? Choose deliberately
-

2 Setup and Your First Call

- 2.1 Create the project
 - 2.2 Configure a model
 - 2.3 Your first ChatClient call
 - 2.4 Structured output: getting objects, not prose
-

PART II — THE RECIPES

3 Recipe 1 — The Conversational Agent

- 3.1 Giving the agent a job description
 - 3.2 Adding memory
 - 3.3 Conversation IDs and the mistake everyone makes
 - 3.4 Persisting memory beyond a restart
-

4 Recipe 2 — The Tool-Using Agent

- 4.1 Writing the tools
 - 4.2 Wiring the agent
 - 4.3 Watching it reason and act
 - 4.4 Controlling the loop
-

5 Recipe 3 — The Grounded Agent (RAG)

- 5.1 Ingesting your documents
 - 5.2 The one-line version: QuestionAnswerAdvisor
 - 5.3 The tunable version: RetrievalAugmentationAdvisor
 - 5.4 Retrieval as a tool instead of an advisor
-

6 Recipe 4 — The Routing Agent

- 6.1 Classify, then dispatch
 - 6.2 Implementation
 - 6.3 Why this beats one big prompt
-

7 Recipe 5 — The Orchestrator with Parallel Workers

- 7.1 The plan-execute-synthesise shape
 - 7.2 Implementation
 - 7.3 Running workers in parallel safely
-

8 Recipe 6 — The Self-Correcting Agent

- 8.1 Generate, evaluate, refine
 - 8.2 Implementation
 - 8.3 Termination: the part that bites
-

9 Recipe 7 — The Delegating Agent (Subagents)

- 9.1 Why context isolation matters
 - 9.2 Defining subagents in Markdown
 - 9.3 Wiring the orchestrator
 - 9.4 Routing subagents to different models
-

PART III — TAKING IT TO PRODUCTION

10 Choosing the Right Pattern

11 Observability, Guardrails, and Cost

12 Testing Agents

A Appendix: API Quick Reference

B Appendix: Sources and Further Reading

1 What an Agent Actually Is

Before any code: five minutes on what the word means. Nearly every frustrating afternoon with an agent framework traces back to building an agent where a much simpler thing would have worked, or the reverse.

1.1 Three things people call “an agent”

The term covers a spectrum from "one prompt" to "a system that decides its own next step." The distinction that matters is **who decides what happens next** — you, or the model.

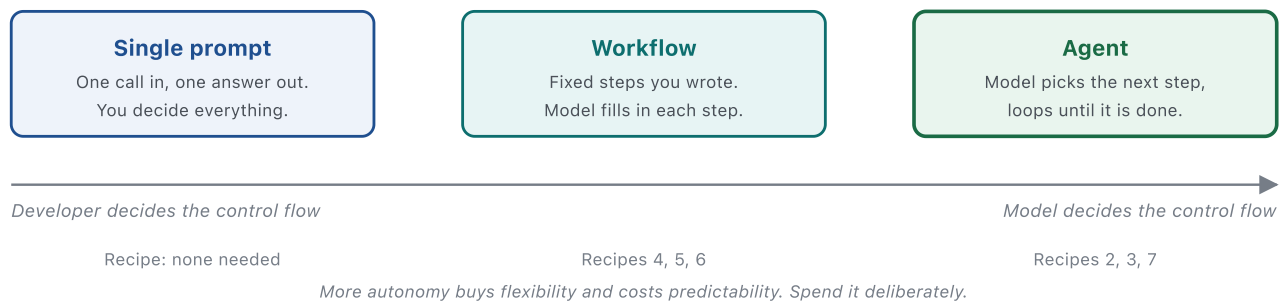


Figure 1.1 — The autonomy spectrum. Recipes in this tutorial sit at different points on this line. Note that the most advanced recipes are not the rightmost ones: a well-built workflow (Recipes 4–6) is often the better engineering choice, and knowing when to stop short of a full agent is most of the skill.

1.2 The agent loop

Strip away the vocabulary and an agent is a `while` loop with a language model inside it. The model is handed a goal, a set of tools, and the history so far; it responds either with a tool call ("I want to look something up") or with a final answer ("here is your result"). If it called a tool, you run it, append the result, and ask again.

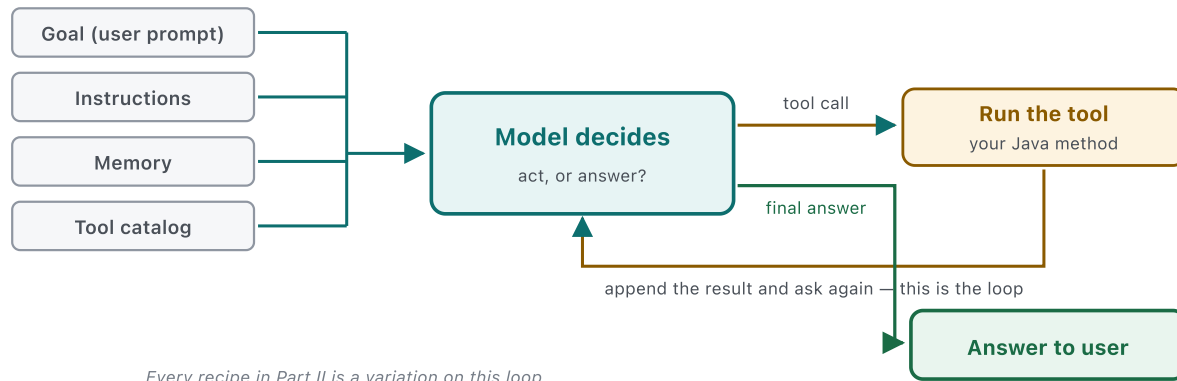


Figure 1.2 — The agent loop. Spring AI runs this loop for you: when you attach tools to a `ChatClient`, it automatically registers a `ToolCallingAdvisor` that detects tool-call responses, invokes your Java method, appends the result, and re-prompts — until the model returns a plain answer.

1.3 Workflow or agent? Choose deliberately

QUESTION	IF YES → WORKFLOW	IF NO → AGENT
Do you know the steps in advance?	Write them as code. Recipes 4–6.	Let the model choose. Recipes 2, 7.
Must the cost and latency be predictable?	A fixed workflow makes exactly N model calls.	An agent loop makes an unknown number.
Is a wrong step expensive or irreversible?	Keep control in your code.	Only with confirmation and guardrails (§11).
Do you need to explain what happened?	A workflow's trace is your own call graph.	An agent's trace is a model transcript.

RULE OF THUMB

Start at the simplest recipe that could work, and move right along Figure 1.1 only when you hit a concrete limitation. Every recipe in Part II ends with a "where this breaks down" section that names the limitation motivating the next one.

2 Setup and Your First Call

This chapter gets a working project on your machine and a first response from a model. If you already have a Spring AI project running, skim to §2.4 — structured output is used by several later recipes.

2.1 Create the project

Start from start.spring.io with **Spring Web** and the AI starter for your provider, or add these to an existing Boot 4 project. Spring AI 2.0.1 requires Spring Boot 4.0.x or 4.1.x and Java 17+.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>2.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-openai</artifactId>
  </dependency>
</dependencies>
```

Swap the starter for your provider — `spring-ai-starter-model-anthropic`, `spring-ai-starter-model-ollama`, and so on. Everything in this tutorial is provider-portable: only this one dependency and the properties below change. All Spring AI artifacts are on Maven Central; no extra repositories are required.

2.2 Configure a model

```
# application.properties
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.options.model=gpt-4o
spring.ai.openai.chat.options.temperature=0.2
```

KEEP THE KEY OUT OF THE REPOSITORY

Read it from an environment variable as shown, never a literal in `application.properties`. A committed API key is the single most common way AI project credentials leak.

2.3 Your first ChatClient call

`ChatClient` is the API you will use for everything that follows. Spring Boot auto-configures a `ChatClient.Builder`; inject it and build.

```
package com.example.agents;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class HelloAgent {

    @Bean
    CommandLineRunner hello(ChatClient.Builder builder) {
        ChatClient chatClient = builder.build();

        return args -> {
            String answer = chatClient.prompt()
                .user("In one sentence: what is Spring AI?")
                .call()
                .content();

            System.out.println(answer);
        };
    }
}
```

TRY IT

Run `./mvnw spring-boot:run`. You should see a one-sentence answer printed to the console. If you get an authentication error, your API key is not reaching the application — check the environment variable is exported in the same shell.

Three details worth noticing in that snippet, because every later recipe builds on them:

- `.prompt()` starts a request; `.user(...)` sets the user message.
- `.call()` is synchronous. `.stream()` returns a `Flux<String>` instead, for token-by-token output.
- Nothing is sent until you ask for the result — `.content()` here. `.call()` on its own does not invoke the model.

2.4 Structured output: getting objects, not prose

Agents that make decisions need to read the model's answer, not just show it to a human. `.entity(Class)` gets you a typed Java object instead of a `String` — Spring AI generates a schema from the type, asks the model to conform to it, and deserialises the result.

```
record Classification(String category, double confidence, String reasoning) {}
```

```
Classification c = chatClient.prompt()  
    .user("Classify this support ticket: 'I was charged twice this month'")  
    .call()  
    .entity(Classification.class);
```

```
System.out.println(c.category()); // billing
```

Records are ideal here: they are concise, immutable, and their component names become the schema's field names. Recipes 4, 5, and 6 all use this to let one model call drive Java control flow.

WHEN THE MODEL RETURNS SOMETHING UNPARSEABLE

Structured output is a request, not a guarantee. For higher reliability, ask Spring AI to use the provider's native structured-output mode and to validate the result against the schema:

```
.entity(Classification.class, spec -> spec  
    .useProviderStructuredOutput()  
    .validateSchema());
```

3 Recipe 1 — The Conversational Agent

DIFFICULTY



YOU'LL LEARN System instructions, chat memory, conversation scoping

USE WHEN The agent needs to hold a coherent multi-turn conversation but does not yet need to *do* anything

NEW APIS `defaultSystem`, `ChatMemory`, `MessageWindowChatMemory`, `MessageChatMemoryAdvisor`

The simplest thing worth calling an agent: something with a persistent role and a memory of what was said. Everything later in this tutorial assumes these two pieces.

3.1 Giving the agent a job description

The **system prompt** is the agent's standing instructions — who it is, what it may do, how it should behave when unsure. Set it once with `defaultSystem` so every request carries it.

1 Define the agent as a bean

```
@Configuration
public class SupportAgentConfig {

    @Bean
    ChatClient supportAgent(ChatClient.Builder builder) {
        return builder
            .defaultSystem("""
                You are a support assistant for Acme Orders.
                Be concise and factual. If you do not know something,
                say so and offer to escalate – never guess an order detail.
            """)
            .build();
    }
}
```

A good system prompt states the **role**, the **tone**, and above all the **failure behaviour**. That last line matters more than it looks: without an explicit instruction about what to do when it does not know, a model's default is to produce something plausible.

3.2 Adding memory

A `ChatClient` is stateless. Ask it "what did I just say?" and it has no idea. Memory is added as an **advisor** — an interceptor in the request pipeline that reads prior messages in and writes new ones out.

2 Add a memory advisor

```
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.memory.MessageWindowChatMemory;
import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;

@Bean
ChatMemory chatMemory() {
    return MessageWindowChatMemory.builder()
        .maxMessages(20)    // the default; tune to your context budget
        .build();
}

@Bean
ChatClient supportAgent(ChatClient.Builder builder, ChatMemory chatMemory) {
    return builder
        .defaultSystem("You are a support assistant for Acme Orders. ...")
        .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
        .build();
}
```

`MessageWindowChatMemory` keeps a sliding window of the most recent messages. Two behaviours are worth knowing: it **never evicts** `SystemMessage`s, so the agent cannot forget its own instructions, and it evicts on **turn boundaries** — a user message and everything that followed it go together, so the window never ends up holding a tool call whose result was trimmed away.

3.3 Conversation IDs and the mistake everyone makes

Memory is keyed by a **conversation ID**. You must supply it on every call:

3 Pass a conversation ID per call

```
String reply = supportAgent.prompt()
    .user("My name is Dana.")
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
    .call()
    .content();

// later, same conversationId:
String reply2 = supportAgent.prompt()
    .user("What is my name?")
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
    .call()
    .content();    // "Your name is Dana."
```

TWO MISTAKES TO AVOID

Forgetting the parameter. There is no default conversation ID. Omit `ChatMemory.CONVERSATION_ID` and you get an `IllegalArgumentException` at runtime, not a silent fallback.

Hard-coding it. A constant like `"default"` works perfectly in development with one user and merges every user's conversation into one shared history in production. Derive it on the server from the authenticated user and session — `currentUser.getId() + ":" + session.getId()` — never from a value the client can choose.

3.4 Persisting memory beyond a restart

The default repository is in-memory: history dies with the process. Swap in a persistent `ChatMemoryRepository` and nothing else changes.

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-chat-memory-repository-jdbc</artifactId>
</dependency>
```

```
@Bean
ChatMemory chatMemory(JdbcChatMemoryRepository repository) {
    return MessageWindowChatMemory.builder()
        .chatMemoryRepository(repository)
        .maxMessages(20)
        .build();
}
```

Repository implementations ship for JDBC (PostgreSQL, MySQL/MariaDB, SQL Server, HSQLDB, Oracle), Cassandra, Neo4j, MongoDB, and Redis, each as a `spring-ai-starter-model-chat-memory-repository-*` starter.

A TRAP FOR RECIPE 2

The JDBC and MongoDB repositories **silently drop tool-call messages** — `AssistantMessage`s carrying tool calls and `ToolResponseMessage`s are filtered out rather than stored. That is harmless for a pure chat agent, but once you add tools in Recipe 2 the persisted history will have gaps where the agent's actions should be. Check your chosen repository's support before relying on replayable tool history.

WHERE THIS BREAKS DOWN

This agent can talk about orders but cannot look one up. Everything it "knows" was baked into its weights or typed into the conversation. That is Recipe 2.

4 Recipe 2 — The Tool-Using Agent

DIFFICULTY



YOU'LL LEARN Tools, the reason-act loop, controlling autonomy

USE WHEN The agent must fetch live data or take action in your systems

NEW APIS `@Tool`, `@ToolParam`, `defaultTools`, `ToolContext`

This is the step where a chatbot becomes an agent. Give the model a catalogue of Java methods it may call, and it will decide — per request, from the wording of your descriptions — which ones to invoke and in what order.

4.1 Writing the tools

A tool is an ordinary Spring bean method annotated with `@Tool`. The `description` is not documentation — it is the *only* thing the model reads when deciding whether to call it.

1 Write a tool class

```
package com.example.agents.tools;

import org.springframework.ai.tool.annotation.Tool;
import org.springframework.ai.tool.annotation.ToolParam;
import org.springframework.stereotype.Component;

@Component
public class OrderTools {

    private final OrderRepository orders;

    public OrderTools(OrderRepository orders) {
        this.orders = orders;
    }

    @Tool(name = "get_order_status",
        description = "Look up the current status and tracking number of one order. \"\"\" +\n"
            + "Call this whenever the customer asks where their order is.")
    public OrderStatus getOrderStatus(
        @ToolParam(description = "The order ID from the customer's confirmation email.")
        String orderId) {

        return orders.findById(orderId)
            .map(o -> new OrderStatus(o.id(), o.status(), o.trackingNumber()))
            .orElse(new OrderStatus(orderId, "NOT_FOUND", null));
    }

    @Tool(name = "list_recent_orders",
```

```

        description = "List the customer's most recent orders, newest first. Call this "" +
            "when the customer refers to an order but cannot give an order ID.")
    public List<OrderSummary> listRecentOrders(
        @ToolParam(description = "How many orders to return. Defaults to 5.", required = false)
        Integer limit) {

        return orders.findRecent(limit == null ? 5 : limit);
    }

    public record OrderStatus(String orderId, String status, String trackingNumber) {}
    public record OrderSummary(String orderId, String status, Instant placedAt) {}
}

```

DESCRIPTION QUALITY IS THE WHOLE GAME

Notice that both descriptions say *when* to call the tool, not just what it does. That second sentence is what lets the model choose `list_recent_orders` when the customer says "my last order" and `get_order_status` when they quote an ID. Vague descriptions produce an agent that calls the wrong tool or none at all.

4.2 Wiring the agent

2 Attach the tools

```

@Bean
ChatClient supportAgent(ChatClient.Builder builder, ChatMemory chatMemory, OrderTools orderTools) {
    return builder
        .defaultSystem("""
            You are a support assistant for Acme Orders.
            Use the provided tools to look up real order data –
            never state an order status you have not retrieved from a tool.
            """)
        .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
        .defaultTools(orderTools)
        .build();
}

```

That is the entire integration. Spring AI generates a JSON schema from each method signature, sends the catalogue with every request, and auto-registers a `ToolCallingAdvisor` that runs the loop from Figure 1.2.

REINFORCE TOOL USE IN THE SYSTEM PROMPT

The added instruction — *never state an order status you have not retrieved* — is doing real work. Tool descriptions tell the model what is available; the system prompt tells it how much it is allowed to improvise when tools are available.

4.3 Watching it reason and act

Ask the agent something that requires two steps and it will chain the calls itself:

User: "Where's my last order?"

```
→ model calls list_recent_orders(limit=1)
← [{"orderId":"4471","status":"SHIPPED","placedAt":"2026-09-02T10:12:00Z"}]
→ model calls get_order_status(orderId="4471")
← {"orderId":"4471","status":"SHIPPED","trackingNumber":"1Z999AA10123456784"}
```

Agent: "Your most recent order, 4471, shipped and is tracking as 1Z999AA10123456784."

Nobody wrote that two-step plan. The model derived it from the two tool descriptions. This is the **ReAct** pattern — reason, act, observe, repeat — and in Spring AI it is what you get by default from attaching tools.

SEE THE CALLS

Turn on request logging to watch this happen instead of inferring it:

```
logging.level.org.springframework.ai.chat.client.advisor=DEBUG
logging.level.org.springframework.ai.tool=DEBUG
```

4.4 Controlling the loop

An agent that can act is an agent that can act *wrongly*, repeatedly, at your expense. Three controls matter from day one.

CONTROL	HOW
Hide sensitive inputs	A tenant ID or user ID must never be a tool parameter the model fills in — it would be free to invent one. Pass it out of band with <code>ToolContext</code> : add a <code>ToolContext</code> parameter to the method and supply values with <code>.toolContext(Map.of("tenantId", tenant))</code> . It is excluded from the schema the model sees.
Separate read from write	Keep destructive tools in a separate class and attach them only on the code paths that should have them. A read-only support agent and an order-management agent can share a <code>ChatClient.Builder</code> and differ only in <code>.defaultTools(...)</code> .
Return errors as data	Expected outcomes ("no such order") should be normal return values, so the model can relay them. Reserve exceptions for genuine failures; their message is sent back to the model, so write it as guidance and never leak internal detail such as hostnames or SQL.

WHERE THIS BREAKS DOWN

This agent handles anything your tools cover. But ask it "what is our refund policy for opened electronics?" and there is no tool to call — the answer lives in a hundred pages of policy documents. That is Recipe 3.

5 Recipe 3 — The Grounded Agent (RAG)

DIFFICULTY



YOU'LL LEARN

Vector stores, retrieval advisors, grounding answers in your documents

USE WHEN

The knowledge the agent needs lives in documents, not in APIs

NEW APIS

`VectorStore`, `QuestionAnswerAdvisor`, `RetrievalAugmentationAdvisor`

Retrieval-Augmented Generation means: before answering, search your own content and put the relevant pieces into the prompt. The agent then answers from evidence you supplied rather than from memory of its training data.

5.1 Ingesting your documents

Documents are chunked, embedded as vectors, and stored. You need an embedding model and a `VectorStore`; for a first run, the simple in-memory store avoids standing up a database.

1 Add a vector store and embedding model

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-vector-store-advisor</artifactId>
</dependency>
```

2 Load documents at startup

```

import org.springframework.ai.document.Document;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;

@Bean
CommandLineRunner ingest(VectorStore vectorStore) {
    return args -> {
        List<Document> docs = List.of(
            new Document("Opened electronics may be returned within 14 days for store credit.",
                Map.of("source", "returns-policy", "section", "electronics")),
            new Document("Unopened items may be returned within 30 days for a full refund.",
                Map.of("source", "returns-policy", "section", "general"))
        );

        // Split long documents into chunks that fit the embedding model's window.
        vectorStore.add(new TokenTextSplitter().apply(docs));
    };
}

```

METADATA NOW SAVES YOU LATER

Attach metadata (`source` , `section` , and in multi-tenant systems a `tenant`) at ingestion time. You cannot filter on what you did not store, and re-embedding a corpus to add a field is slow and expensive.

5.2 The one-line version: QuestionAnswerAdvisor

3 Attach the advisor

```

import org.springframework.ai.vectorstore.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.vectorstore.SearchRequest;

@Bean
ChatClient policyAgent(ChatClient.Builder builder, VectorStore vectorStore) {
    return builder
        .defaultSystem("Answer only from the provided policy context. If the context does not " +
            "cover the question, say so.")
        .defaultAdvisors(QuestionAnswerAdvisor.builder(vectorStore)
            .searchRequest(SearchRequest.builder()
                .similarityThreshold(0.75)
                .topK(6)
                .build())
            .build())
        .build();
}

```

That is a working RAG agent. On each request the advisor embeds the user's question, retrieves the closest chunks, and injects them into the prompt before the model sees it.

KNOB	EFFECT
<code>topK</code>	How many chunks to retrieve. Too few and the answer is missing context; too many and the real evidence is diluted by noise. Start at 4–6.
<code>similarityThreshold</code>	Minimum closeness to include a chunk. Raising it returns less, but more relevant, context — and can legitimately return nothing.
<code>FILTER_EXPRESSION</code>	Restrict retrieval per request — essential for multi-tenancy. Pass at call time with <pre>.advisors(a -> a.param(QuestionAnswerAdvisor.FILTER_EXPRESSION, "tenant == 'acme'")) .</pre>

5.3 The tunable version: RetrievalAugmentationAdvisor

When basic retrieval is not good enough, `RetrievalAugmentationAdvisor` exposes the pipeline as replaceable modules: transform the query, retrieve, then augment the prompt.

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-rag</artifactId>
</dependency>
```

```
import org.springframework.ai.rag.advisor.RetrievalAugmentationAdvisor;
import org.springframework.ai.rag.retrieval.retrieve.VectorStoreDocumentRetriever;
import org.springframework.ai.rag.query.transformer.RewriteQueryTransformer;
import org.springframework.ai.rag.generation.augmentation.ContextualQueryAugmenter;
```

```
Advisor ragAdvisor = RetrievalAugmentationAdvisor.builder()
    // Rewrite conversational questions into standalone search queries:
    // "and what about opened ones?" → "return policy for opened electronics"
    .queryTransformers(RewriteQueryTransformer.builder()
        .chatClientBuilder(builder.build().mutate())
        .build())
    .documentRetriever(VectorStoreDocumentRetriever.builder()
        .vectorStore(vectorStore)
        .similarityThreshold(0.70)
        .topK(5)
        .build())
    // Refuse to answer when nothing was retrieved, rather than guessing.
    .queryAugmenter(ContextualQueryAugmenter.builder()
        .allowEmptyContext(false)
        .build())
    .build();
```

ALLOWEMPTYCONTEXT IS A SAFETY SWITCH

Left at its default (`false`), a question with no matching documents produces a polite refusal. Set it to `true` and the model answers from general knowledge — which, for a policy agent, is exactly the failure mode RAG existed to prevent. Change it only when a fallback answer is genuinely better than "I don't know."

5.4 Retrieval as a tool instead of an advisor

An advisor retrieves on *every* request — including "thanks, bye." Making retrieval a **tool** instead lets the agent decide when a search is warranted, and combines naturally with the tools from Recipe 2.

```
@Component
public class PolicyTools {

    private final VectorStore vectorStore;

    public PolicyTools(VectorStore vectorStore) { this.vectorStore = vectorStore; }

    @Tool(name = "search_policies",
        description = "Search company policy documents for returns, refunds, shipping and "" +
            "warranty rules. Call this before answering any policy question.")
    public List<String> searchPolicies(
        @ToolParam(description = "A natural-language description of the policy to find.")
        String query) {

        return vectorStore.similaritySearch(
            SearchRequest.builder().query(query).topK(4).build())
            .stream()
            .map(Document::getText)
            .toList();
    }
}
```

APPROACH	CHOOSE IT WHEN
Advisor (§5.2–5.3)	Nearly every request is a question about the corpus — a documentation Q&A bot. Simpler, one retrieval per turn, predictable cost.
Tool (§5.4)	The agent does several kinds of work and only sometimes needs the corpus. It can also search repeatedly with reformulated queries when the first result is unhelpful.

WHERE THIS BREAKS DOWN

One agent now has order tools, policy search, memory, and a long system prompt covering every situation. As you add a fourth and fifth capability, the instructions start to conflict and tool selection gets less reliable. The fix is to stop making one agent do everything — Recipe 4.

6 Recipe 4 — The Routing Agent

DIFFICULTY



YOU'LL LEARN Classification-driven dispatch, workflows as plain Java

USE WHEN Requests fall into distinct categories that each want different instructions or tools

NEW APIS None — structured output plus ordinary Java control flow

The first **workflow** recipe. Here you take control back from the model: one call classifies the request, then your code — not the model — decides which specialist handles it.

THESE PATTERNS ARE CODE YOU WRITE, NOT CLASSES YOU IMPORT

Recipes 4, 5, and 6 implement the agentic workflow patterns from Spring AI's *Building Effective Agents* reference. Spring AI deliberately does not ship `RoutingWorkflow` or `OrchestratorWorkersWorkflow` as framework types — they are example implementations, because the useful version is always shaped by your domain. What follows is a complete implementation you can paste in and adapt.

6.1 Classify, then dispatch

The shape is: **one cheap classification call** → **a Java switch** → **one specialist call**. Two model calls, both bounded, and the routing decision is inspectable and testable on its own.

6.2 Implementation

1 Define the classification result

```
public record RouteDecision(  
    String category,           // billing | technical | general  
    double confidence,  
    String reasoning           // keep this: it is your debugging trail  
) {}
```

2 Write the router

```
@Component  
public class RoutingAgent {  
  
    private final ChatClient chatClient;  
    private final Map<String, String> routes;
```

```

public RoutingAgent(ChatClient.Builder builder) {
    this.chatClient = builder.build();
    this.routes = Map.of(
        "billing", ""
            You are a billing specialist. Explain charges precisely, cite amounts and
            dates, and never speculate about a refund you cannot confirm.
            "",
        "technical", ""
            You are a technical support engineer. Diagnose step by step and ask for the
            one piece of missing information you most need.
            "",
        "general", ""
            You are a customer service representative. Be warm, brief, and route the
            customer onward if the question needs a specialist.
            "");
}

public String handle(String userInput) {
    RouteDecision decision = classify(userInput);

    // Your code decides, not the model. Unknown categories fall back safely.
    String systemPrompt = routes.getOrDefault(decision.category(), routes.get("general"));

    return chatClient.prompt()
        .system(systemPrompt)
        .user(userInput)
        .call()
        .content();
}

private RouteDecision classify(String userInput) {
    return chatClient.prompt()
        .system("""
            Classify the support request into exactly one category:
            billing - charges, invoices, refunds, payment methods
            technical - errors, broken features, setup problems
            general - everything else
            Give your confidence from 0.0 to 1.0 and one sentence of reasoning.
            """)
        .user(userInput)
        .call()
        .entity(RouteDecision.class);
}
}

```

3 Use the low-confidence signal

```

RouteDecision decision = classify(userInput);

if (decision.confidence() < 0.6) {
    // Ambiguous input: fall back to the generalist, or ask a clarifying question,
    // or hand to a human. This branch is why we asked for confidence at all.
    return escalateToHuman(userInput, decision);
}

```

6.3 Why this beats one big prompt

BENEFIT	DETAIL
Sharper instructions	Each specialist prompt is short and single-purpose. One combined prompt trying to cover billing, technical, and general advice gets long, and long prompts blur.
Cheaper	Classification is a tiny call that can use a small fast model. Only the specialist call needs your most capable one.
Testable	<code>classify()</code> is a pure function from text to a category. You can build a table of 200 real tickets and assert the routing without ever invoking a specialist.
Observable	When a customer complains, <code>reasoning</code> and <code>confidence</code> tell you exactly why the request went where it did.

WHERE THIS BREAKS DOWN

Routing assumes every request has *one* right handler, chosen from a list you wrote in advance. "Write release notes for this changelog, and a customer email, and a migration guide" is not one category — it is three sub-tasks you did not enumerate. That is Recipe 5.

7 Recipe 5 — The Orchestrator with Parallel Workers

DIFFICULTY



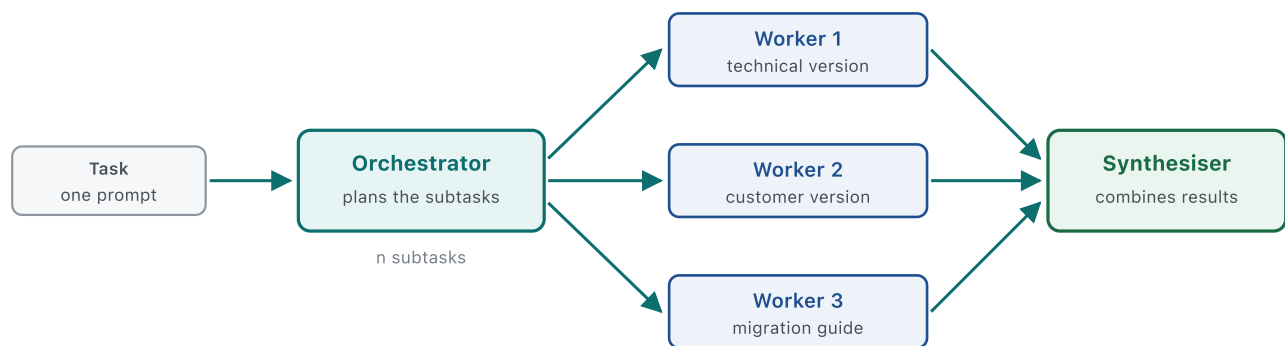
YOU'LL LEARN Dynamic task decomposition, parallel model calls, result synthesis

USE WHEN The subtasks cannot be known in advance, and they are independent of one another

NEW APIS Structured output driving a fan-out; `CompletableFuture` or a virtual-thread executor

Here a first model call **plans** — it decides what the subtasks are — and your code executes that plan across workers running concurrently, then synthesises the results. The plan is dynamic; the execution stays under your control.

7.1 The plan–execute–synthesise shape



Workers run concurrently — wall-clock time is one worker, not n .

Figure 7.1 — Orchestrator–workers. The orchestrator decides *how many* workers there are and what each one does, which is what distinguishes this from Recipe 4's fixed route table. Independence between subtasks is the precondition: if worker 2 needs worker 1's output, this is a chain, not a fan-out.

7.2 Implementation

1 Model the plan and the result

```

public record Subtask(String name, String description) {}

public record Plan(String analysis, List<Subtask> subtasks) {}

public record OrchestratorResult(String analysis, List<String> workerOutputs, String finalOutput) {}

```

2 Write the orchestrator

@Component

```

public class OrchestratorAgent {

    private final ChatClient chatClient;

    public OrchestratorAgent(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    public OrchestratorResult process(String task) {

        // ---- 1. PLAN -----
        Plan plan = chatClient.prompt()
            .system("""
                Break the task into 2-4 independent subtasks that can be done in parallel.
                Each subtask must stand alone – a worker sees only its own description,
                never the other subtasks or their results.
            """)
            .user(task)
            .call()
            .entity(Plan.class);

        // ---- 2. EXECUTE (in parallel) -----
        List<String> outputs;
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            List<CompletableFuture<String>> futures = plan.subtasks().stream()
                .map(subtask -> CompletableFuture.supplyAsync(
                    () -> runWorker(task, subtask), executor))
                .toList();

            outputs = futures.stream().map(CompletableFuture::join).toList();
        }

        // ---- 3. SYNTHESISE -----
        String finalOutput = chatClient.prompt()
            .system("Combine the worker outputs into one coherent deliverable. " +
                "Remove duplication; keep every distinct point.")
            .user("Original task: " + task + "\n\nWorker outputs:\n" + String.join("\n---\n", outputs))
            .call()
            .content();

        return new OrchestratorResult(plan.analysis(), outputs, finalOutput);
    }

    private String runWorker(String originalTask, Subtask subtask) {
        return chatClient.prompt()
            .system("You are a specialist worker. Do exactly your assigned subtask, nothing more.")
            .user("""
                Overall goal: %s

                Your subtask (%s): %s
            """)
    }
}

```

```

        """formatted(originalTask, subtask.name(), subtask.description()))
        .call()
        .content();
    }
}

```

3 Run it

```

OrchestratorResult result = orchestrator.process(
    "Produce release notes for the 2.0 upgrade: a technical changelog, " +
    "a customer-facing announcement, and a migration guide.");

System.out.println(result.analysis()); // why it split the work this way
System.out.println(result.finalOutput()); // the combined deliverable

```

7.3 Running workers in parallel safely

CONCERN	WHAT TO DO
Rate limits	Four workers means four concurrent API calls. Providers rate-limit per key, and a burst returns 429s. Cap concurrency with a bounded executor or a semaphore rather than fanning out whatever number the model happened to plan.
Unbounded plans	Nothing stops the model planning 40 subtasks. Validate the plan before executing: reject or truncate above a limit you set. Treat the plan as untrusted input, because it is.
Partial failure	<code>CompletableFuture::join</code> propagates the first failure and abandons the rest. Decide explicitly: fail the whole task, or synthesise from the workers that did succeed. The second is usually better — wrap each worker in a try/catch returning a placeholder.
Cost	This pattern makes <code>n + 2</code> model calls per request. Budget for the worst case, not the average, and consider a smaller model for workers than for the orchestrator.

VIRTUAL THREADS SUIT THIS PERFECTLY

Worker calls are almost entirely network wait. `newVirtualThreadPerTaskExecutor()` on Java 21+ gives you cheap concurrency without sizing a pool. On Java 17, use a small fixed pool instead — and either way close the executor, as the try-with-resources above does.

WHERE THIS BREAKS DOWN

The orchestrator produces output in one pass. Nothing checks whether it is any good, and for tasks with clear quality criteria — correct code, a translation that preserves tone — one pass is often not enough. That is Recipe 6.

8 Recipe 6 — The Self-Correcting Agent

DIFFICULTY



YOU'LL LEARN Generator/evaluator separation, iterative refinement, loop termination

USE WHEN Quality is judgeable against criteria, and a second pass reliably improves it

NEW APIS None new — two roles, one loop, strict bounds

Two model roles working against each other: a **generator** that produces a candidate and an **evaluator** that critiques it. The critique feeds the next attempt. This is where agents start to visibly improve their own output — and where runaway loops start to cost real money.

8.1 Generate, evaluate, refine

The evaluator must be a **separate call with a separate prompt**. Asking one call to "write this and then check your work" produces a model that grades its own homework generously; a fresh call with only the criteria and the artefact is a far harsher critic.

8.2 Implementation

1 Model the evaluation

```
public record Evaluation(
    boolean pass,
    int score,           // 1-10
    List<String> issues, // specific, actionable
    String suggestion
) {}

public record RefinedResult(String solution, int iterations, List<String> history) {}
```

2 Write the loop

```
@Component
public class SelfCorrectingAgent {

    private static final int MAX_ITERATIONS = 3;

    private final ChatClient chatClient;

    public SelfCorrectingAgent(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }
}
```

```

}

public RefinedResult refine(String task, String criteria) {
    List<String> history = new ArrayList<>();
    String candidate = generate(task, null, null);

    for (int i = 1; i <= MAX_ITERATIONS; i++) {
        Evaluation evaluation = evaluate(task, criteria, candidate);
        history.add("Iteration %d - score %d: %s"
            .formatted(i, evaluation.score(), String.join("; ", evaluation.issues())));

        if (evaluation.pass()) {
            return new RefinedResult(candidate, i, history);
        }
        // Feed the critique back in as the basis for the next attempt.
        candidate = generate(task, candidate, evaluation);
    }

    // Budget exhausted. Return the best we have - never loop forever.
    return new RefinedResult(candidate, MAX_ITERATIONS, history);
}

private String generate(String task, String previous, Evaluation critique) {
    String user = (previous == null)
        ? task
        : ""
            Task: %s

            Your previous attempt:
            %s

            It was rejected for these reasons:
            %s

            Produce a corrected version that fixes every issue listed.
            """".formatted(task, previous, String.join("\n- ", critique.issues()));

    return chatClient.prompt()
        .system("You are an expert practitioner. Produce only the deliverable, no commentary.")
        .user(user)
        .call()
        .content();
}

private Evaluation evaluate(String task, String criteria, String candidate) {
    return chatClient.prompt()
        .system("""
            You are a demanding reviewer. Judge the submission ONLY against the criteria.
            Be specific: every issue must name what is wrong and where.
            Set pass=true only if the submission fully meets every criterion.
            """)
        .user("""
            Task: %s

            Criteria: %s

            Submission:
            %s
            """".formatted(task, criteria, candidate))
        .call()
        .entity(Evaluation.class);
}

```

```
}  
}
```

3 Run it

```
RefinedResult result = agent.refine(  
    "Write a Java method that merges two sorted int arrays.",  
    "Must handle null and empty inputs, run in O(n+m), and have Javadoc.");  
  
result.history().forEach(System.out::println);  
// Iteration 1 – score 6: no null handling; missing Javadoc  
// Iteration 2 – score 9:  
System.out.println("Converged after " + result.iterations());
```

8.3 Termination: the part that bites

ALWAYS BOUND THE LOOP

A loop that exits only when the evaluator says `pass` can run forever, because an evaluator can always find something to improve. `MAX_ITERATIONS` is not a nicety — it is the difference between a bounded cost and an open-ended bill. Bound every agent loop by iteration count, and in production by wall-clock time and token budget as well.

SYMPTOM	CAUSE AND FIX
Never passes	Criteria are subjective or unreachable. Make them checkable ("has Javadoc") rather than aspirational ("is elegant"), and accept the best-scoring attempt when the budget runs out.
Passes immediately, output is poor	The evaluator is too lenient. Push the reviewer prompt harder and require it to list issues before deciding pass — judging then concluding beats concluding then justifying.
Oscillates between two versions	Fixing issue A reintroduces issue B. Include the full history in the refinement prompt, not just the last critique, so the generator can see the cycle.
Cost spikes	Each iteration is two calls on a growing prompt. Cap iterations at 2–3; the first refinement captures most of the gain.

WHERE THIS PATTERN SHINES

Use a deterministic evaluator when one exists. If the artefact is code, *compile it and run the tests*, then feed real compiler and test output back as the critique. A model grading code is a heuristic; a failing test is a fact.

WHERE THIS BREAKS DOWN

Every recipe so far runs in one conversation, so every tool result, retrieved document, and rejected draft accumulates in a single context window. On a long task the window fills with intermediate detail and answer quality degrades. That is Recipe 7.

9 Recipe 7 — The Delegating Agent (Subagents)

DIFFICULTY

●●●●●

YOU'LL LEARN

Context isolation, declarative subagents, per-subagent tool and model policy

USE WHEN

A task is long enough that intermediate detail crowds out the goal

NEW APIS

`spring-ai-agent-utils` : `TaskToolCallbackProvider` , `ClaudeSubagentReferences`

COMMUNITY LIBRARY, NOT SPRING AI CORE

This recipe uses `spring-ai-agent-utils` from the `spring-ai-community` organisation (group id `org.springframework.ai`) — not an official Spring AI module. It requires Spring AI 2.0.0-M2 or later. Everything in Recipes 1–6 uses core Spring AI only; this one adds a dependency, so weigh that before adopting it.

9.1 Why context isolation matters

Ask one agent to "review the authentication module and summarise the risks" and its context window fills with file contents, grep output, and false leads. By the time it writes the summary, the actual goal is buried under thousands of tokens of exploration — and model quality degrades measurably as a window fills with low-signal material.

A **subagent** runs in its own conversation with its own window, does the exploring, and returns only its conclusion. The parent never sees the intermediate mess.

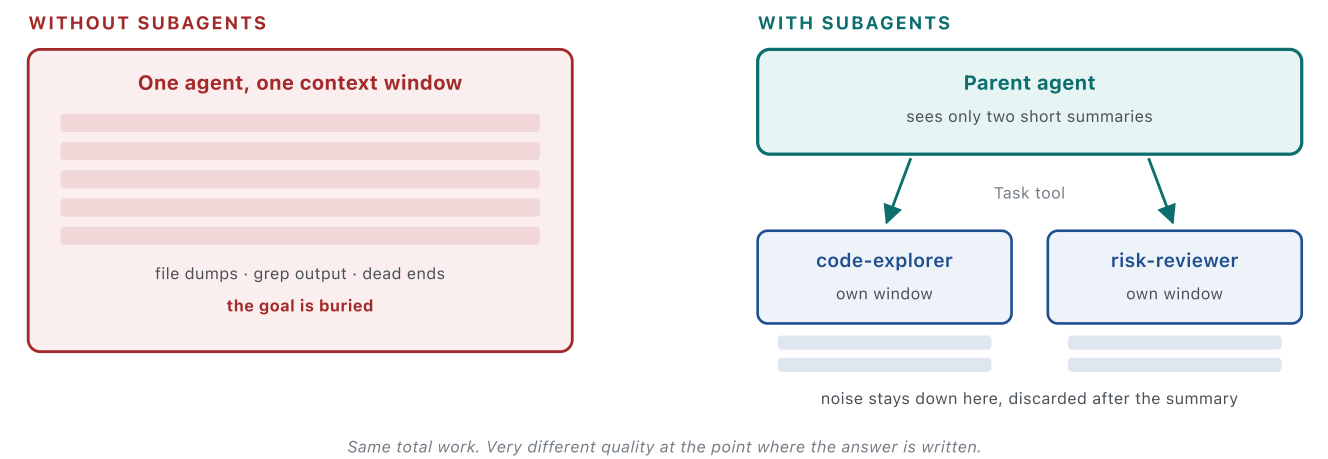


Figure 9.1 — Context isolation. The parent's window holds the goal and two conclusions. Each subagent's window holds the exploration and is then thrown away. This is the same principle as a function call with a narrow return type — the caller does not inherit the callee's local variables.

9.2 Defining subagents in Markdown

Subagents are **declarative**: a Markdown file with YAML frontmatter, no Java required. This is the same format Claude Code uses, so definitions are portable.

1 Add the dependency

```
<dependency>
  <groupId>org.springaicomunity</groupId>
  <artifactId>spring-ai-agent-utils</artifactId>
  <version>0.4.2</version>
</dependency>
```

2 Write a subagent definition

```
<!-- src/main/resources/agents/code-reviewer.md -->
---
name: code-reviewer
description: Expert code reviewer. Use proactively after code is written or changed.
tools: Read, Grep, Glob
disallowedTools: Edit, Write
model: sonnet
---

You are a senior code reviewer focused on correctness and security.

When invoked:
1. Identify what changed.
2. Read only the files you need; do not summarise the whole repository.
3. Report findings ordered by severity, each with a file and line reference.
4. Return a short summary – the caller does not want your exploration notes.
```

FRONTMATTER FIELD	REQUIRED	PURPOSE
name	Yes	Unique identifier, lowercase with hyphens.
description	Yes	How the parent decides to delegate. Written for a model to read, exactly like a tool description.
tools	No	Tools this subagent may use. Inherits all if omitted.
disallowedTools	No	Explicit denials. Above, the reviewer can read but cannot modify anything.
model	No	Which registered model to run on — see §9.4.

THE LAST INSTRUCTION IS THE IMPORTANT ONE

"Return a short summary" is what makes the pattern pay off. A subagent that returns its entire transcript re-imports all the noise into the parent's window and you have gained nothing but latency.

9.3 Wiring the orchestrator

3 Give the parent the Task tool

```
import org.springframework.ai.agent.tools.task.TaskToolCallbackProvider;

@Configuration
public class DelegatingAgentConfig {

    @Bean
    ChatClient delegatingAgent(ChatClient.Builder builder) {

        var taskTools = TaskToolCallbackProvider.builder()
            .chatClientBuilder("default", builder)
            .subagentReferences(
                ClaudeSubagentReferences.fromRootDirectory("src/main/resources/agents"))
            .build();

        return builder
            .defaultSystem("""
                You coordinate specialists. For any non-trivial investigation, delegate to a
                subagent with the Task tool rather than exploring yourself. Synthesise their
                summaries into one answer.
            """)
            .defaultToolCallbacks(taskTools)
            .build();
    }
}
```

4 Delegate

```
String answer = delegatingAgent.prompt()
    .user("Review the authentication module and tell me the top three risks.")
    .call()
    .content();
```

At startup the provider scans the `agents/` directory and builds a registry of each subagent's name and description. At request time the parent sees the Task tool with that registry, picks a subagent by matching the request against those descriptions, and invokes it in a fresh conversation. Only the subagent's final message returns.

9.4 Routing subagents to different models

Not every subtask deserves your most expensive model. Register several builders by name, and the `model` field in each Markdown file selects one.

```
var taskTools = TaskToolCallbackProvider.builder()
    .chatClientBuilder("default", sonnetBuilder) // balanced default
    .chatClientBuilder("haiku", haikuBuilder) // fast and cheap: file triage, formatting
    .chatClientBuilder("opus", opusBuilder) // slow and strong: security review
    .subagentReferences(
        ClaudeSubagentReferences.fromRootDirectory("src/main/resources/agents"))
    .build();
```

This is where hierarchical agents earn their cost: a cheap model does the grepping, an expensive one does the judging, and the parent pays for neither transcript.

GOING CROSS-PROCESS: A2A

Subagents here run in your JVM. To delegate to an agent owned by *another team or service*, the same library integrates the **Agent2Agent (A2A)** protocol, an open standard for agent-to-agent discovery and messaging — the natural next step once delegation needs to cross a network boundary.

WHERE THIS BREAKS DOWN

Nowhere technical — this is the most capable pattern in the tutorial. It breaks down *economically*: every delegation is a full agent run, so a system that delegates reflexively is slow and expensive. Delegate when a subtask is genuinely large enough that isolating its context is worth an extra round trip, and not by default.

10 Choosing the Right Pattern

Seven recipes, one decision. Work down this table and stop at the first row that matches — the recipes get more powerful as you descend, and more expensive to run and debug.

IF YOUR SITUATION IS...	USE	BECAUSE
Multi-turn conversation, no external data	Recipe 1 Conversational	Instructions plus memory is all you need; anything more is overhead.
Needs live data or must take action	Recipe 2 Tool-using	Tools are the cheapest way to make an agent useful, and the loop is built in.
Knowledge lives in documents	Recipe 3 RAG	Grounds answers in retrievable evidence instead of model memory.
Distinct request categories, known in advance	Recipe 4 Routing	Short focused prompts beat one prompt covering every case; classification is testable.
Subtasks unknown up front but independent	Recipe 5 Orchestrator	The model plans; your code controls execution and parallelism.
Output quality is judgeable and iteration helps	Recipe 6 Self-correcting	A separate evaluator catches what the generator missed.
Long tasks whose intermediate detail drowns the goal	Recipe 7 Subagents	Context isolation keeps the parent's window clean.

These compose. A production support agent is plausibly Recipe 4 routing into three specialists, one of which is a Recipe 2 tool-using agent with Recipe 3 retrieval, all carrying Recipe 1 memory. Build them one at a time, in that order.

11 Observability, Guardrails, and Cost

11.1 See what the agent is doing

Agents fail in ways stack traces do not capture: the right tool called with wrong arguments, a retrieval that returned nothing, a plan with four subtasks where two were duplicates. You cannot debug what you cannot see.

```
# The minimum worth turning on in development
logging.level.org.springframework.ai.chat.client.advisor=DEBUG
logging.level.org.springframework.ai.tool=DEBUG
```

CAPTURE	WHY IT MATTERS
Every tool call and its arguments	The single most useful signal. Wrong-tool and wrong-argument errors dominate real agent bugs.
Token counts per request	Cost lives here, and a slow context-window leak shows up as steadily rising input tokens.
Iteration counts	For Recipes 5–7: how many workers, how many refinement rounds, how many delegations.
Retrieved document IDs	For Recipe 3: "the answer was wrong" is unactionable; "it retrieved the wrong three chunks" is a fix.

Spring AI emits observations through Micrometer, so the same infrastructure carrying your HTTP and JDBC metrics will carry model calls, token usage, and tool invocations.

11.2 Guardrails

RISK	MITIGATION
Runaway loops	Bound every loop by iterations, wall-clock time, and token budget. Recipe 6's <code>MAX_ITERATIONS</code> is the pattern; apply it to Recipes 5 and 7 too.
Destructive tool calls	Keep read and write tools in separate classes and attach write tools only where they belong. For genuinely irreversible actions, require human confirmation between the model's request and execution.
Prompt injection via retrieved content	A document in your corpus, or a tool result from an external API, can contain instructions aimed at your model. Treat all retrieved text as data. Never let it silently expand the agent's permissions, and never interpolate it into a system prompt.
Leaking internals	Exception messages from tools are sent back to the model and can end up in a user-visible answer. Catch and rewrite them; never surface raw driver or HTTP client messages.
Tenant bleed	Pass tenant and user identity through <code>ToolContext</code> and vector-store filter expressions — never as a tool parameter the model fills in, and never from a client-supplied value.

11.3 Cost and latency

PATTERN	MODEL CALLS PER REQUEST	NOTES
Recipe 1 — Conversational	1	Grows with memory window size, not call count.
Recipe 2 — Tool-using	1 + one per tool round	Unbounded in principle; cap tool iterations.
Recipe 3 — RAG	1 + embedding	Advisor retrieves every turn; the tool variant only when needed.
Recipe 4 — Routing	2	Use a small model for classification.
Recipe 5 — Orchestrator	$n + 2$	n is chosen by the model — validate the plan before executing.
Recipe 6 — Self-correcting	2 per iteration	Bounded only by <code>MAX_ITERATIONS</code> .
Recipe 7 — Subagents	1 + a full agent run per delegation	The most expensive; route cheap subtasks to cheap models.

12 Testing Agents

Models are non-deterministic, which makes agent testing feel impossible. It is not — you just have to test the deterministic parts deterministically, and treat the model-facing parts as a separate, smaller suite.

12.1 Three layers

LAYER	DETERMINISTIC?	WHAT TO TEST
Tools	Yes — fully	Ordinary unit tests. A tool is a Java method: cover its success, empty, and error paths with no model involved at all. This is where most of your coverage belongs.
Workflow logic	Yes, with a stub	Recipes 4–6 are plain Java around model calls. Stub the <code>ChatClient</code> to return fixed classifications or evaluations and assert your routing, fan-out, and termination behave — including that the loop stops at <code>MAX_ITERATIONS</code> .
Model behaviour	No	A small suite of real calls checking the model chooses sensibly. Assert properties, not exact strings.

12.2 Testing model behaviour without asserting prose

```
@Test
void routesBillingQuestionsToBillingSpecialist() {
    // Assert the DECISION, not the wording of the final answer.
    RouteDecision decision = routingAgent.classify("I was charged twice this month");

    assertThat(decision.category()).isEqualTo("billing");
    assertThat(decision.confidence()).isGreaterThan(0.7);
}

@Test
void agentLooksUpOrderRatherThanGuessing() {
    // Assert the observable side effect: the tool was actually called.
    supportAgent.prompt()
        .user("Where is order 4471?")
        .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, "test-1"))
        .call()
        .content();

    verify(orderRepository).findById("4471");
}
```

ASSERT PROPERTIES, NOT TEXT

`assertThat(answer).isEqualTo("Your order shipped.")` will fail the first time the model rephrases. Assert instead that a tool was called, that a category was chosen, that the answer contains the tracking number, or that a refusal happened when the context was empty. Those hold across rewordings and across model upgrades.

A Appendix: API Quick Reference

A.1 ChatClient

CALL	PURPOSE
<code>ChatClient.builder(chatModel)</code>	Create a builder programmatically; or inject the auto-configured <code>ChatClient.Builder</code> .
<code>.defaultSystem(String)</code>	Standing instructions applied to every request.
<code>.defaultAdvisors(Advisor...)</code>	Memory, RAG, logging — the request pipeline.
<code>.defaultTools(Object...)</code>	Attach <code>@Tool</code> beans; auto-registers the tool-calling loop.
<code>.prompt().user(String)</code>	Start a request and set the user message.
<code>.call()</code> / <code>.stream()</code>	Synchronous, or reactive <code>Flux</code> .
<code>.content()</code>	The response as a <code>String</code> . Nothing is sent to the model until this (or <code>entity</code>) is called.
<code>.entity(Class<T>)</code>	Structured output as a typed object. Use records.
<code>.toolContext(Map)</code>	Data for tools that the model must not see or supply.

A.2 Memory

TYPE	NOTES
<code>MessageWindowChatMemory</code>	Default strategy. Sliding window, default 20 messages, never evicts system messages, evicts on turn boundaries.
<code>MessageChatMemoryAdvisor</code>	Attaches memory to a <code>ChatClient</code> .
<code>ChatMemory.CONVERSATION_ID</code>	Required on every call. No default; omitting it throws.
<code>JdbcChatMemoryRepository</code> and peers	Persistence for JDBC, Cassandra, Neo4j, MongoDB, Redis. JDBC and MongoDB silently drop tool-call messages.

A.3 Tools and RAG

TYPE	NOTES
@Tool , @ToolParam	org.springframework.ai.tool.annotation . Description decides when the model calls it.
ToolContext	Out-of-band data, excluded from the model-visible schema.
QuestionAnswerAdvisor	org.springframework.ai.vectorstore.advisor , artifact spring-ai-vector-store-advisor . Simple RAG.
RetrievalAugmentationAdvisor	org.springframework.ai.rag.advisor , artifact spring-ai-rag . Modular RAG pipeline.
VectorStoreDocumentRetriever	Retriever with threshold, topK, and filter expressions.
ContextualQueryAugmenter	allowEmptyContext(false) refuses rather than guesses.

B Appendix: Sources and Further Reading

PRIMARY SOURCES FOR THIS TUTORIAL

- [spring.io/projects/spring-ai](#) — project overview, current release line (2.0.1), supported providers.
- [ChatClient API reference](#) — builder, advisors, structured output, tool attachment.
- [Chat Memory reference](#) — `MessageWindowChatMemory`, repositories, conversation IDs.
- [Tool Calling reference](#) — `@Tool`, `@ToolParam`, `ToolContext`, the tool-calling loop.
- [RAG reference](#) — advisors, retrievers, query transformers, augmenters.
- [Building Effective Agents reference](#) — the five workflow patterns behind Recipes 4–6.
- [Building Effective Agents with Spring AI \(blog\)](#) — the original write-up of those patterns.
- [Spring AI Agentic Patterns: Subagent Orchestration](#) — the Task tool and Markdown subagents used in Recipe 7.
- [spring-ai-examples](#) — runnable reference implementations of the workflow patterns.

SPRING AI RECIPES — CRAIG WALLS

[habuma.com/springairecipes](#) is a growing collection of short, focused Spring AI recipes. Ones that extend this tutorial directly:

RECIPE	EXTENDS
Building a Text-Based Chat Loop Around ChatClient	Recipe 1 — a real interactive loop.
Persisting Chat Memory with JDBC · 3 More Ways to Persist Chat Memory · Enabling Long-Term Memory	Recipe 1 — memory beyond a restart.
Reasoning and Acting (ReAct) · Efficient Tool Use · Explaining Tool Selection	Recipe 2 — tool-use mechanics and why the model chose a tool.
Essential RAG · Tool-Based RAG · Better RAG Results with Reranking / Hybrid Search / HyDE · Filtering RAG Results with Metadata · Making RAG Conversation-Aware	Recipe 3 — the full RAG progression.
Building a Graph-Based Agentic Workflow (LangGraph4j) · Adding a Loop to a Graph-Based Workflow · Adding Human-in-the-Loop	Recipes 4–6 — workflows as explicit graphs.

RECIPE	EXTENDS
Defining Claude-Style Subagents · Guiding Agent Behavior with Skills · Reusing Agent Behavior with SkillsJars · Agentic Planning with TodoWriteTool	Recipe 7 — delegation, skills, and planning.
Enabling Agent-to-Agent Communication with A2A · Invoking A2A Sub-Agents with TaskTool	Recipe 7 — delegation across process boundaries.
Applying Semantic Guardrails · Applying Input Guardrails with SafeGuardAdvisor · Safeguarding LLM Output	§11.2 — guardrails.
Logging LLM Requests and Responses · Better LLM Request/Response Logging with ToolCallAdvisor	§11.1 — observability.
Asking Questions to the User	§11.2 — human-in-the-loop confirmation.
Semantic Caching (with Redis / any vector store) · Using More than One LLM	§11.3 — cost control.
Enabling Resilient Structured Output	§2.4 — when <code>.entity()</code> needs to be reliable.

VERSION NOTE

This tutorial targets **Spring AI 2.0.1** on Spring Boot 4.0/4.1. Spring AI's agent and MCP APIs have moved quickly across recent versions; Recipe 7 in particular depends on the community `spring-ai-agent-utils` library (0.4.2), which versions independently of Spring AI itself. Check the reference documentation for your exact versions before treating any class or package name here as final.